



LICEO BICENTENARIO MARIO BERTERO CEVASCO

PROGRAMACIÓN PYTHON — 4TO MEDIO 2026

PROFESOR: DIEGO VARGAS PALOMINOS

GUÍA DE ESTUDIO

CLASE 24A — FUNCIONES

`def` · parámetros · `return`

Objetivo de la clase: Construir funciones propias con `def` que resuelvan una tarea puntual, con claridad.

Propósito: *La claridad es explicar algo de forma que la otra persona entienda sin tener que adivinar ni preguntar de nuevo. Hoy la practicamos nombrando funciones y parámetros que dejan clara su tarea, sin que haga falta leer lo que hacen por dentro.*

Nombre: _____

Fecha: _____

Vocabulario nuevo

Cuatro palabras que vas a necesitar hoy

- **Función:** un bloque de código con nombre propio que agrupa una tarea, para no repetirla completa cada vez que se necesita.
- **Parámetro:** una variable que la función espera recibir. Se define al escribir `def nombre(parametro):`
- **Argumento:** el valor real que le entregas a la función cuando la llamas — ocupa el lugar del parámetro.
- **return:** entrega un valor de vuelta a quien llamó la función y corta la ejecución ahí mismo. Sin **return**, la función devuelve `None`.

El problema del kiosco

El kiosco del liceo cobra normal a todos, pero a quienes están inscritos en el Centro de Estudiantes (CEE) les da un descuento fijo de \$300 en cualquier compra. El problema es la fila del recreo: llegan decenas de estudiantes seguidos, cada uno con un producto distinto, y para cada uno hay que hacer el mismo cálculo — preguntar si está inscrito y, si corresponde, restar los \$300 — todo mientras la fila sigue creciendo. Con tanta repetición y apuro, ya se han cobrado de más a socios del CEE y de menos a quienes no lo son.

Fíjate que el cálculo en sí no cambia entre una persona y otra: son los mismos pasos, una y otra vez, y solo cambian el precio y si la persona está inscrita. Justo ese tipo de tarea — la misma lógica aplicada repetidamente con datos distintos — es la que hoy resolvemos construyendo una función: la definimos una sola vez, y la usamos todas las veces que aparezca alguien en la fila.

Definir con `def` y parámetros. `def nombre_funcion(parametro1, parametro2):` abre el bloque; el cuerpo va indentado debajo. Los parámetros son variables que la función espera recibir.

```
def descuento_socio(precio, es_socia):  
    if es_socia:  
        return precio - 1500  
    return precio
```

Idea clave: definir la función no la ejecuta — solo queda "guardada la receta" hasta que alguien la llama.

Lllamar la función: argumentos. Llamar la función es escribir su nombre con paréntesis y los valores reales (argumentos) que ocupan el lugar de los parámetros.

```
precio_final = descuento_socio(8000, True)  
print("Paga:", precio_final)
```

Idea clave: parámetro es el nombre en la definición; argumento es el valor que se manda al llamar.

return: devolver un resultado. `return` entrega un valor de vuelta a quien llamó la función y termina la ejecución en ese punto. Sin `return`, la función devuelve `None`.

```
def saluda(nombre):  
    print("Hola,", nombre)  
  
resultado = saluda("Camila")  
print(resultado)
```

```
>> Hola, Camila  
>> None
```

Idea clave: `print()` muestra algo en pantalla, pero solo `return` deja el valor disponible para usarlo después.

Resolvamos el problema del kiosco — paso a paso

Paso 1 — Definimos la función.

```
def calcula_cobro(precio, inscrita_cee):  
    # precio e inscrita_cee son los PARAMETROS  
    if inscrita_cee:  
        return precio - 300    # return corta la funcion aqui  
    return precio              # si no esta inscrita, se devuelve tal cual
```

Definir la función todavía no calcula nada — solo queda "guardada la receta", lista para usarse con cualquier estudiante que llegue a la fila.

Paso 2 — La llamamos con argumentos reales.

```
precio_producto = int(input("¿Cuánto cuesta el producto? "))  
inscrita = input("¿Está inscrita en el CEE? (si/no) ") == "si"  
  
total = calcula_cobro(precio_producto, inscrita)  
# precio_producto e inscrita son los ARGUMENTOS: ocupan el  
# lugar de precio e inscrita_cee al llamar la funcion  
print("Debe pagar:", total)
```

Paso 3 — Por qué `return` y no `print()` dentro de la función.

Si `calcula_cobro` imprimiera el resultado en vez de devolverlo con `return`, la variable `total` quedaría en `None` — tendrías el valor en pantalla, pero no disponible para seguir usándolo (como sumarlo, guardarlo o compararlo).

Programa completo, armado:

```
def calcula_cobro(precio, inscrita_cee):
    if inscrita_cee:
        return precio - 300
    return precio

precio_producto = int(input("¿Cuánto cuesta el producto? "))
inscrita = input("¿Está inscrita en el CEE? (si/no) ") == "si"

total = calcula_cobro(precio_producto, inscrita)
print("Debe pagar:", total)
```

Ejemplo 1	Ejemplo 2
Entrada: 1200 si	Entrada: 800 no
Salida: Debe pagar: 900	Salida: Debe pagar: 800

Errores típicos a evitar

Error	Qué ocurre	Cómo corregirlo
Llamar la función sin paréntesis (<code>descuento_socio</code>)	No ejecuta nada, solo referencia a la función	Agregar <code>()</code> con los argumentos
Confundir <code>print()</code> interno con <code>return</code>	La función "muestra" el resultado pero no lo entrega — la variable que la recibe queda en <code>None</code>	Usar <code>return</code> si el valor se necesita después
Esperar que una variable definida dentro de la función exista afuera	<code>NameError</code> al intentar usarla fuera	Devolver el valor con <code>return</code> y guardarlo en una variable afuera
Escribir código después de un <code>return</code> esperando que se ejecute	Ese código nunca corre	Recordar que <code>return</code> corta la función ahí mismo

Práctica Independiente

Revisa en tu celular: cuando termines cada ejercicio, revisa tu respuesta en el celular — la solución completa está en el cuaderno Solucionario subido a Classroom.

Ejercicio 1 — Huerto escolar. El huerto del liceo organiza brigadas de cosecha y junta los tomates en cajones de 12 para venderlos en la feria del sábado. Cada brigadista anota cuántos tomates cosechó, pero armar los cajones a mano y contar los que sobran ya generó más de un error.

El programa debe:

- Definir una función que reciba la cantidad de tomates cosechados y devuelva cuántos cajones completos de 12 se pueden armar.

- Pedir la cantidad cosechada.
- Llamar la función y mostrar el resultado.

Ejemplo 1	Ejemplo 2
Entrada: 50	Entrada: 137
Salida: Cajones completos: 4	Salida: Cajones completos: 11

Escribe tu programa aquí:

Ejercicio 2 — Club Deportivo de Isla de Maipo. El Club Deportivo cobra una cuota mensual fija, y algunos socios llegan atrasados varios meses. Quieren calcular rápido cuánto deben pagar en total.

El programa debe:

- Definir una función con dos parámetros (cuota mensual, meses atrasados) que devuelva el total a pagar.
- Pedir esos dos datos.
- Llamar la función y mostrar el total.

Ejemplo 1	Ejemplo 2
Entrada: 5000 3	Entrada: 8000 2
Salida: Total a pagar: 15000	Salida: Total a pagar: 16000

Escribe tu programa aquí:

Ejercicio 3 — Peña de la Vendimia. La Peña de la Vendimia, la fiesta típica de Isla de Maipo, cobra entrada general en la puerta, pero los niños menores de 12 años entran liberados y las personas de 65 años o más pagan la mitad. El organizador anota mal los cobros cuando hay mucha gente esperando y quiere automatizarlo.

El programa debe:

- Definir una función que reciba la edad de la persona y el precio general, y devuelva cuánto debe pagar.
- Aplicar las tres categorías: menores de 12 (liberado), 65 o más (mitad de precio), el resto (precio general).
- Pedir la edad de un asistente y el precio general, llamar la función, y mostrar cuánto debe pagar.

Ejemplo 1	Ejemplo 2
Entrada: 8 5000	Entrada: 70 5000
Salida: Debe pagar: 0	Salida: Debe pagar: 2500

Escribe tu programa aquí:

Ejercicio 4 — Desafío: Escuela de Rock. *(opcional)*

La profesora de la Escuela de Rock de la comuna lleva el registro de horas de práctica de cada estudiante y necesita saber, para cualquiera de ellos, cuántas horas le faltan para llegar a las 20 horas mínimas del semestre. Como revisa a varios estudiantes seguidos antes de cada clase, quiere repetir el cálculo tantas veces como estudiantes tenga enfrente, sin cerrar el programa entre uno y otro.

El programa debe:

- Definir una función que reciba las horas que un estudiante ya practicó y devuelva cuántas horas le faltan para llegar a 20 (nunca un número negativo).
- Repetir el cálculo para distintos estudiantes hasta que la profesora escriba "listo" en vez de un nombre.
- Para cada estudiante, pedir su nombre y sus horas practicadas, llamar la función, y mostrar cuántas horas le faltan.

El programa imprime:

```
¿Nombre del estudiante? (o "listo" para terminar) Fernanda
¿Cuántas horas practicó? 14
A Fernanda le faltan 6 horas.
¿Nombre del estudiante? (o "listo" para terminar) Benjamín
¿Cuántas horas practicó? 22
A Benjamín le faltan 0 horas.
¿Nombre del estudiante? (o "listo" para terminar) listo
```

Escribe tu programa aquí:

Vas bien. Cada función que aprendes a construir es una herramienta más que te queda para siempre — sigue así.